Deliverable 42 — Issue 1 integrated into the framework builder + full-app agnosticism re-sweep; Issue 2 deferred

Date: 2026-10-19
Author: CompanyIQ engineering
Commits: `a8302a1` (Part A — builder stability), `a3df49a` (Part B — agnosticism audit doc)
Production SHA (verified live): `a3df49a` — production **app** and **worker** both `SUCCESS` @ 2026-10-19T11:29:39Z
Verdict impact: Yes→No = **0**, No→Yes = **0**, Total flips = **0**

---

1. What you asked for

> 1. *Defer Issue 2 instead of doing it now.*
> 2. *The Issue 1 solution should be implemented at the level of the **framework builder**, not based on a specific framework — integrated into the LLM builder — validated by creating a new framework.*
> 3. *Review the whole application to ensure it is topic- and company-agnostic, with no requirements or components tailored to any specific topic or company.*

This deliverable records all three outcomes. There is **no runtime verdict change** in any of them: Part A is build-time/advisory, Part B is documentation, and Issue 2 was not touched.

---

2. Issue 2 — deferred (recorded, not implemented)

Issue 2 (human-analyst holdout to validate the B5 context-trim on-flip gate) is **deferred at your request**. No code, config, env flag, or batch was run for it. The on-flip gate for B5 remains deliberately **unarmed** and `CONTEXT_COST_POLICY` stays in `shadow` (owner decision A, ledger 2026-10-19) — exactly as before. Nothing about Issue 2's eventual implementation is pre-empted or blocked by this turn's work.

---

3. Issue 1 — now hardened at the builder level (every new framework, by design)

3.1 Approach: audit-first, then close the one real gap

Before writing anything, the existing builder (`server/lib/framework-v2/`) was audited for reproducibility machinery. It already had a substantial amount:

- **C11** (`rules.ts`) — the ERROR-level degree-word gate that blocks a build when a measure's verdict criteria hinge on unquantified degree words.
- **C12** (`rules.ts`) — the INFO-level companion signal.

- `edit-proposer.ts` — residual-instability / ambiguous-criteria detection that proposes rewrites, driven by **multiple scoring runs**.
- `edit-applier.ts` — ambiguity-removal when applying proposed edits.
- `quality-metrics.ts` — a cross-run reliability dimension (kappa / agreement).
- End-to-end per-cell stability was already surfaced: `analyzer.ts:3576` maps self-consistency agreement to a `confidence` level (High/Med/Low, plus `[Self-consistency N/M]`) → `api.ts` → ResultsPage CSV.

The single genuine gap: all of the above required *actual scoring runs* to detect instability. There was **no build-time, zero-runs-needed check** that inspects a newly-authored measure's own verdict logic and warns the author *before any company is ever scored*. That is the gap this work closes — and it closes it in the builder, so it applies to **every** framework created from now on, not to any one framework.

3.2 What was added (Part A, commit `a8302a1` — 6 files, +467/−8)

- **New pure module `server/lib/framework-v2/verdict-stability.ts` (~191 lines).** Exposes `classifyMeasureStability`, which classifies each measure as `stable` or `borderline` purely from the measure's own text/criteria — **no scoring runs required**. It is completely topic-agnostic: it composes the *same* ambiguity predicates already used by C11/C12, which are now `export`ed from `rules.ts` (zero logic change in `rules.ts` — only the export surface widened, so the one definition is shared, never duplicated).
- **Wired into the existing builder review surface.** `measure-design-diagnostic.ts` — the builder's pre-test diagnostic that authors already see — now emits an **advisory** `verdict_instability_risk` finding, one per reason, for borderline measures. This is the integration point you asked for: it lives inside the builder's own existing feedback surface, so creating a new framework exercises it automatically.
- **Advisory, not a new hard gate.** The new finding is informational. The sole ERROR-level build gate remains **C11** (unchanged); **C12** remains INFO. We did not newly block any build — that would change builder behaviour beyond scope and risk false-blocking legitimate frameworks. Authors are *warned early*; they are not newly *stopped*.
- **One shared guidance constant.** `COUNTABLE_REWRITE_GUIDANCE` is defined once and consumed by both the new build-time path and the existing `edit-proposer.ts` post-hoc multi-run rationale — no duplicated wording.

3.3 How you validate it

Create a new framework in Framework Builder V2 with a deliberately vague verdict criterion (e.g. a measure that turns on an unquantified degree word like "substantial" or "adequate"). The builder's measure-design diagnostic should surface a `verdict_instability_risk` advisory for that measure *before* you run any scoring — pointing at the specific ambiguity and the countable-rewrite guidance. A tightly-quantified measure should produce no such advisory. Because the check is framework-agnostic, the same behaviour holds for any topic.

3.4 Why this is verdict-invariant

The classifier runs at **build/author time** and emits **advisory text only**. It does not touch the analyzer, the scoring prompt, the evidence pack, self-consistency, or any lever. No existing framework's scores change. **Yes→No = 0, No→Yes = 0.**

---

4. Agnosticism re-sweep (Part B, commit `a3df49a` — documentation only)

A full-app re-sweep of `server/`, `shared/`, and `client/src/` was run for topic-specific literals (AI/ML/LLM/AGI/GPAI, ESG/climate/nature families), company names / tickers / ISINs from the original AI-governance roster, and any hardcoded framework-id / measure-id / measure-name strings that **gate** behaviour. The verbatim greps and the full findings table are recorded in `GENERICNESS_AUDIT.md` §5.

Result: no code fixes required. The sweep found **no genuine hidden single-topic or single-company gate**. Every topic/company literal encountered falls into one of four benign classes, each reviewed and kept:

1. **Multi-topic illustrative prompt examples** (e.g. `intake-prompt.ts`, `topic-lexicon.ts` list TNFD/TCFD/SBTi *and* LLM/AGI/GPAI side by side) — these teach the model to emit the right acronyms for *whatever* topic it is given; removing them weakens every topic equally.
2. **Regression comments and test fixtures** (including the Prudential ISIN motivation comment for the R5e domain bug) — not production gates.
3. **Universal framework-pillar / disclosure-document / document-format constructs** — e.g. `passage-retrieval.ts` routes on pillar names like Governance/Strategy/Risk/Metrics that every disclosure framework (TCFD/TNFD/ISSB) shares; discovery queries are built from `framework.name` at runtime and explicitly instructed *not* to emit generic ESG queries unless that is the topic; `CSRD_TABLE_NOTATION_NOTE` is a PDF-format parsing aid.
4. **Scoring infrastructure** — model/provider names (deepseek, openai, anthropic, mistral, gemini, …) identify which model to call, not entities being scored.

Topic-specific behaviour throughout the app is derived **at runtime** from the framework's own `topicTerm` / `topicDescription` / `topicSynonyms` / terminology and from the company record — never baked into shared code. The app and Framework Builder V2 are **confirmed topic-agnostic at the gating level**.

4.1 One residual risk — flagged for your decision, deliberately NOT changed

The scoring system prompt (`analyzer.ts:569` and `:725`) opens with the persona *"You are an expert ESG/governance analyst scoring corporate disclosures…"*. For ESG-family topics (including AI **governance**, the product's first target) this is on-topic; for a hypothetical non-ESG framework it is a mild framing **prior**.

- It is **not a gate.** The real subject is injected at runtime via `Topic: ${topicDescription}` and the anti-inference rules derive from `framework.topicTerm`. The persona cannot pin the output to any one topic or company.
- It was **left unchanged on purpose.** Editing the live scoring prompt is a **verdict-affecting runtime change** — on genuinely borderline measures a reworded persona could flip cells — which is (a) outside this turn's build-time/advisory + agnosticism-refactor scope, and (b) impossible to validate without running a scoring batch, which is prohibited here.
- **Recommended future handling (your call):** generalise the persona to a neutral *"expert analyst scoring corporate disclosures against a structured assessment framework"*, then validate verdict stability on a frozen corpus before shipping. **This needs an explicit go-ahead and an accepted scoring-batch cost** — tell me if you want it scoped as its own change.

---

5. Gates (all green)

Gate	Result
`tsc --noEmit` (server)	62 pre-existing errors before → **62 after** — 0 new from changed files (the +1 vs an earlier “61” note is pre-existing drift on `main`, not from this work)
Framework-builder test suite	**370 / 370 pass**
Client build	No client code touched → no `vite build` required
Verdict impact	**Yes→No = 0, No→Yes = 0, Total flips = 0**
Deploy parity	Local HEAD == origin/main == production app == production worker == **`a3df49a`**

---

6. Constraint compliance

1. **Code tested == code in production.** HEAD == origin/main == both production Railway services == `a3df49a`; the builder suite ran against exactly that tree.
2. **Yes→No / No→Yes totals in summary tables.** 0 / 0 / 0 — stated above; the work is build-time/advisory (Part A) and documentation (Part B), verdict-invariant by construction.
3. **Framework-level, not company/topic-specific.** The stability classifier is a pure topic-agnostic module in the builder; the agnosticism sweep confirmed 0 hidden gates.
4. **Relevant text never dropped.** Part A emits advisory flags only; Part B is docs. No retrieval, trim, or B5 change — B5 stays in `shadow`.
5. **Phase 2 — no human review of thousands of companies.** The builder hardening is automatic at author time; no per-result human review was added.